

Innføring av nytt system for arbeidstid og ressursplanlegging

Case – Trondheim Renholdsverk

Usman Ghafoorzai

Teknologier og tekniske avklaringer

Versjon <1.0>

Revisjonshistorie

Dato	Versjon	Beskrivelse	Forfatter
09/09/2026	1.0	Første utkast til teknologier og tekniske avklaringer for caseoppgaven.	Usman Ghafoorzai

Innholdsfortegnelse

1. Formål og avgrensning
2. Tekniske avklaringer
3. Aktuelle teknologier og integrasjonsmåter
4. Prinsipper for teknologivalg
5. Teknisk spike / PoC
6. Foreløpig teknisk retning

1. Formål og avgrensning

Formålet er å vise hvordan jeg ville undersøkt det tekniske mulighetsrommet før arkitekturen låses. Casebeskrivelsen oppgir ikke konkrete leverandører, API-er eller integrasjonsmekanismer. Teknologiene under er derfor kandidater som må vurderes mot leverandørstøtte, kravene i virksomheten og resultatene fra kartleggingen.

Målet er ikke å velge mest mulig teknologi, men den enkleste og mest forvaltbare løsningen som gir sikker, sporbar og robust datautveksling mellom ERP-systemet og arbeidstid- og ressursplanleggingssystemet.

2. Tekniske avklaringer

Før teknologivalg ville jeg avklart følgende med leverandørene og relevante fagområder:

Tema	Hva jeg vil avklare	Hvorfor
Grensesnitt	Hvilke støttede integrasjonsgrensesnitt finnes: API, fil, event/webhook eller batch?	Avgrenser hvilke integrasjonsmåter som faktisk er realistiske.
Oppdateringsbehov	Hvilke data må være oppdatert raskt, og hvilke kan overføres periodisk?	Avgjør om behovet er hendelsesdrevet, synkront eller egnet for batch.
Autentisering og autorisasjon	Hvordan sikres system-til-system-tilgang, og hvilke rettigheter kan begrenses?	Nødvendig for sikker tilgang til ansatt- og arbeidstidsdata.
Dataformat og datamodell	Hvilke objekter, felter, identifikatorer og formater støttes?	Gir grunnlag for mapping mellom ERP og tredjepartssystemet.
Testmiljø	Finnes sandbox/testmiljø, testdata og dokumenterte eksempler?	Gjør det mulig å verifisere integrasjonen før produksjon.
Feil og drift	Hvordan rapporteres feil, og hvilke mekanismer finnes for retry, logging og overvåking?	Viktig for robusthet, sporbarhet og forvaltning.
Dataomfang	Trenger tredjepartssystemet f.eks. lønnsarter, og trenger ERP avviks- eller rutedata?	Hindrer unødvendig datautveksling og uklart dataeierskap.

3. Aktuelle teknologier og integrasjonsmåter

Valg av teknologi avhenger av hva de to leverandørene faktisk støtter. Følgende er aktuelle kandidater:

Kandidat	Når det kan passe	Forbehold
REST API + JSON	Når systemene tilbyr stabile API-er og data bør utveksles relativt raskt.	Naturlig kandidat, men ikke et forhåndsvalg.
Event / webhook	Når endringer bør pushes videre når de skjer, for eksempel relevante endringer i masterdata.	Krever at leverandøren støtter hendelser og at feil/retry håndteres.
Fil / batch	Når data ikke er tidskritiske eller leverandørens støttede løsning er periodisk filutveksling.	Kan være enklere og mer robust enn sanntidsintegrasjon for enkelte flyter.
CSV / XML / JSON	Mulige dataformater avhengig av leverandørgrensesnittet.	Format velges ut fra støtte og datamodell, ikke preferanse alene.
OAuth 2.0 / klientautentisering	Eksempel på system-til-system-sikkerhet dersom leverandøren støtter det.	Endelig mekanisme må følge leverandørens sikkerhetsmodell.
Integrasjonsfunksjon / integrasjonsplattform	Ved behov for mapping, validering, flere flyter, retry, sentral logging eller løsere kobling.	Bør bare innføres dersom den gir reell verdi sammenlignet med direkte integrasjon.

3.1 Direkte integrasjon eller integrasjonsfunksjon?

Jeg ville først vurdert direkte leverandørstøttet integrasjon dersom den dekker behovet på en sikker og forvaltbart måte. En egen integrasjonsfunksjon er aktuell dersom prosjektet trenger mer kontroll over transformasjon, feilhåndtering, logging, overvåking eller flere integrasjonsflyter.

4. Prinsipper for teknologivalg

- Velg leverandørstøttede mekanismer fremfor egenutviklede omveier.
- Velg den enkleste løsningen som dekker funksjonelle og ikke-funksjonelle krav.
- Unngå unødvendig tett kobling mellom systemene slik at endringer hos én leverandør blir håndterbare.
- Utveksle bare data mottakssystemet faktisk trenger.
- Sørg for sporbarhet, feilhåndtering og mulighet for overvåking i produksjon.
- Vurder sikkerhet, personvern, driftskostnad og forvaltbart sammen med funksjonalitet.

5. Teknisk spike / PoC

Før endelig arkitektur og integrasjonsmetode låses ville jeg gjennomført en liten teknisk spike i leverandørenes testmiljø. Poenget er å verifisere sentrale antakelser med representative flyter, ikke å bygge hele løsningen.

Test	Hva som verifiseres
Flyt A – ERP → nytt system	Synkronisere én testansatt og ett prosjekt/aktivitet til tredjepartssystemet.
Flyt B – nytt system → ERP	Returnere et godkjent arbeidsgrunnlag med ansattreferanse, dato/periode, timer og prosjekt-/aktivitetsinformasjon.
Verifikasjon	Kontrollere autentisering, datamapping, feilrespons, sporbarhet og hva som skjer ved duplikat eller midlertidig feil.

6. Foreløpig teknisk retning

Teknologien bør ikke låses før leverandørgrensesnitt, dataomfang og driftsbehov er avklart. Hvis begge systemer tilbyr gode og stabile grensesnitt, vil direkte integrasjon være første kandidat. Hvis det er behov for transformasjon, flere flyter, sentral feilhåndtering, retry, logging eller løsere kobling, vil en integrasjonsfunksjon kunne være hensiktsmessig.

Den endelige løsningen velges etter teknisk discovery og en liten PoC, slik at arkitekturvalget bygger på dokumenterte behov og verifiserte tekniske muligheter.